

Member 6

Code viva practice

Gunathilake P.G.K.I. | IT25103600

Guest feedback

Major function + Rating summary and complaint status filtering

Your 45-second introduction

I own guest feedback. My React feature defines the user-facing fields and uses shared form and table components. The Spring Boot module validates requests, enforces department and customer access, and persists typed entities through JPA. My supporting function is rating summary and complaint status filtering. I can demonstrate the core workflow and its validation; the guide lists the advanced work still pending.

1. Why use React, Spring Boot and SQL Server?

React manages the interactive UI. Spring Boot exposes the API and owns validation, authorization and business rules. SQL Server provides durable relational storage. A browser never connects directly to the database.

2. What happens after Save record is clicked?

The form creates a JSON payload. api.js sends POST for creation or PUT with a version for an update. Spring authenticates the account, validates the entity, applies module rules, writes the record and audit entry in a transaction, then returns JSON. React reloads the list.

3. What is your work and what is shared?

I own the feature folder, Java package, tests and these documents for feedback. Shared ownership covers Workbench, the API client, security, audit support, application configuration and theme. I should explain the shared code I use without claiming to have individually authored all of it.

4. Is this production-ready or exactly 75% complete?

No. It is a defined core-workflow milestone intended for the 75% evaluation. Completion against the lecturer's rubric must be checked separately. Evaluation users, manual cross-module references and external integration setup remain limitations.

Explain your business rules

5. How are reviews and complaints distinguished?

The kind field accepts REVIEW or COMPLAINT. Both have a rating and message; status and response provide the complaint workflow.

6. Where is the rating range enforced?

The entity uses Min(1) and Max(5) with NotNull. The backend rejects invalid values even if the request bypasses the form.

7. Can a customer write a staff response?

No. The server replaces customer-supplied status with OPEN and preserves the old response. On creation it clears the response.

8. When can customers edit feedback?

Only while the existing record is OPEN. Once staff move it into review, customer updates are refused by the server.

Show it in the source

Open FeedbackController.java and point to validate() and beforeDelete(). For rules implemented elsewhere, open the module's extra controller or React component. Explain the rule in ordinary language before reading syntax.

Quick memory check

Close the answer sheet. Name the permitted role, the most important rejected input, the database record type, and the response the customer sees after an error.

Explain the edge cases

9. Why require a response for RESOLVED?

A resolution should explain the action taken. The controller rejects a resolved status with a blank response.

10. How is the average rating computed?

The UI sums visible workspace record ratings and divides by the record count. Staff see all feedback, while a customer sees only their own. Both reviews and complaints contribute.

11. Who removes duplicate or inappropriate feedback?

Only the FEEDBACK role. The shared controller enforces staff-only deletion and records an audit entry in the same transaction.

12. Is every review verified against an attended wedding?

No. A booking reference is required text, but attendance verification and public moderation are remaining work.

Show it in the source

Open FeedbackController.java and point to validate() and beforeDelete(). For rules implemented elsewhere, open the module's extra controller or React component. Explain the rule in ordinary language before reading syntax.

Quick memory check

Close the answer sheet. Name the permitted role, the most important rejected input, the database record type, and the response the customer sees after an error.

Trace the code, not just the screen

13. What does this version check protect?

```
if (!Objects.equals(r.version, old.version)) {
    throw new ResponseStatusException(
        HttpStatus.CONFLICT, "Record changed. Refresh and try again.");
}
```

Two staff members may open the same record. After one saves, the stored version changes. The second user's older version is rejected so an old screen cannot silently erase the first update. The JPA `@Version` field also participates in database concurrency control.

14. Why validate in both UI and backend?

The UI gives immediate feedback but can be bypassed. `@Valid` triggers entity constraints on the server, while `validate()` checks module-specific business rules. Never trust a hidden button as authorization.

15. What is a JPA repository?

The interface extends `JpaRepository` with the entity type and Long ID. Spring supplies operations such as `findAll`, `findById`, `saveAndFlush` and `delete`. Hibernate maps entity state to SQL; we do not concatenate user input into SQL queries.

16. Why are record changes transactional?

A record change and its audit entry should succeed together. `@Transactional` commits them as one unit, or rolls back when validation/persistence fails. A frontend message cannot guarantee persistence; the API and database result determine success.

Read these files in this order

Feedback.java -> FeedbackRepository.java -> FeedbackController.java -> shared/CrudController.java -> shared/Access.java -> frontend/src/features/feedback/index.jsx -> frontend/src/shared/Workbench.jsx.

Practice: trace one POST payload field from its HTML input to the Java entity and database column, then back to the list response.

Practice under evaluation conditions

17. A customer sees an empty table. Is the database empty?

Not necessarily. The list filters by stored owner for customer accounts. Check the signed-in username, then use the correct staff role to inspect the department records. Do not remove ownership checks to make the demo look populated.

18. A request returns 400, 401, 403 or 409. Explain.

400: invalid input or business rule. 401: missing/incorrect authentication. 403: authenticated account lacks permission. 409: stale version or conflicting database constraint. Read the API message and explain the correct recovery.

19. How do you know your module works?

Run FeedbackTest to exercise its API using MockMvc and an isolated test database. The suite tests required input, role restrictions, CRUD and stale updates, plus module-specific cases. Then demonstrate the real UI and repeat persistence checks on SQL Server.

20. How do six branches avoid conflicts?

All branches start from one shared foundation. Each adds different frontend, backend, test and document paths. The registry discovers modules, so members do not each edit a central imports list or route file. Shared changes are coordinated once.

Three hands-on drills

- Change one label inside your own index.jsx, run the frontend build, and demonstrate that no shared file needed editing.
- Send invalid input through the UI. Identify the entity constraint or controller rule that rejects it, and state the expected HTTP status.
- Open the most relevant test, explain its arrange/action/assert sequence, then run it. Describe what a failing assertion would mean.

Final answer to prepare

My next work is: Verified attendance checks, public review moderation, and automated complaint escalation. I would implement each step with a clear acceptance test and verify the integrated SQL Server workflow before calling it complete.

Suggested rehearsal: 45 seconds introduction, 3 minutes working demo, 2 minutes code trace, 2 minutes failure cases, 30 seconds remaining work. Use these model answers to learn the code; do not memorize claims you cannot demonstrate.